

Capítulo 1: Fundamentos del Aprendizaje Automático

Universidad de Salamanca

Facultad de Ciencias

Programación Avanzada

Diego M. Jiménez-Bravo · dmjimenez@usal.es

- 1 Introducción
- 2 1.1 Historia y evolución
- 3 1.2 Conceptos fundamentales

1 Introducción

2 1.1 Historia y evolución

3 1.2 Conceptos fundamentales

¿Por qué aprender aprendizaje automático?

En la programación **tradicional**, una persona programadora escribe explícitamente todas las reglas que un programa debe seguir. Por ejemplo:

```
si temperatura > 30: encender_aire_acondicionado()
```

Este enfoque funciona bien para problemas simples, pero muchos problemas del mundo real son demasiado complejos para describirlos con reglas fijas: ¿cómo distinguir automáticamente un gato de un perro en una fotografía? ¿Cómo predecir si una transacción bancaria es fraudulenta? ¿Cómo clasificar correos como *spam* o no *spam*?

El aprendizaje automático invierte el paradigma: en lugar de escribir las reglas a mano, se le enseña al programa a *aprender* esas reglas a partir de ejemplos.

Ejemplo: nadie puede escribir a mano una regla que distinga un gato de un perro en una foto, pero un modelo puede *aprenderla* viendo miles de fotos ya etiquetadas como “gato” o “perro”.

1 Introducción

2 1.1 Historia y evolución

3 1.2 Conceptos fundamentales

Definición y el cambio de paradigma

La **Inteligencia Artificial** se define de forma concisa como el esfuerzo por automatizar tareas intelectuales que normalmente realizan los seres humanos. Es un campo amplio que incluye tanto el aprendizaje automático y el aprendizaje profundo, como métodos más antiguos como la **IA simbólica**, que dominó el campo desde los años 50 hasta finales de los 80.

El aspecto más revolucionario del aprendizaje automático es que representa un **nuevo paradigma de programación**: en vez de que una persona escriba las reglas, la máquina analiza los datos de entrada y las respuestas esperadas para inferir cuáles deberían ser esas reglas.

Para que este aprendizaje ocurra hacen falta **tres elementos clave**:

- ➊ Datos de entrada.
- ➋ Ejemplos de la salida esperada (**etiquetas**).
- ➌ Una forma de medir si el sistema lo está haciendo bien (**función de pérdida**).

Raíces matemáticas y pioneros (1805-1943)

Aunque el término “aprendizaje automático” es moderno, muchos de sus conceptos estadísticos se desarrollaron hace más de dos siglos:

- **1805**: Legendre y Gauss publican el **método de mínimos cuadrados**, la forma más temprana de regresión lineal — todavía hoy la base de multitud de modelos.
- **1936**: Ronald Fisher propone el **análisis discriminante lineal**, una de las primeras técnicas para predecir una categoría (valor cualitativo) a partir de datos numéricos.
- **1943**: Warren McCulloch y Walter Pitts presentan el **primer modelo computacional de neurona biológica**: una simplificación matemática de cómo se activa una neurona, germen de las redes neuronales actuales.

El Test de Turing (1950)

En 1950, Alan Turing publicó su artículo fundacional “*Computing Machinery and Intelligence*”, donde se preguntaba: **¿pueden pensar las máquinas?**

Para evitar entrar en el debate filosófico de qué es “pensar”, propuso un experimento práctico conocido como el **Test de Turing**: una persona conversa por escrito con un humano y con una máquina, sin saber cuál es cuál. Si no logra distinguirlos de forma fiable, se dice que la máquina ha superado el test.

Aunque hoy se discute su validez como medida real de inteligencia, marcó durante décadas el horizonte al que aspiraba la IA.



Fuente: Elliott & Fry (1951), dominio público

El nacimiento formal: el seminario de Dartmouth (1956)

La IA cristalizó formalmente como campo de investigación en **1956**, cuando John McCarthy organizó el **seminario de Dartmouth**, un encuentro de varias semanas que reunió a los principales investigadores del momento.

McCarthy propuso una conjetura muy ambiciosa: cada aspecto del aprendizaje o de la inteligencia podía describirse con tal precisión que una máquina llegaría a simularlo.

Durante las siguientes décadas, la mayoría de expertos creyó que la inteligencia de nivel humano se alcanzaría codificando a mano un conjunto masivo de **reglas lógicas explícitas**. Este enfoque, conocido como **IA simbólica**, alcanzó su punto álgido con los **sistemas expertos** de los años 80.

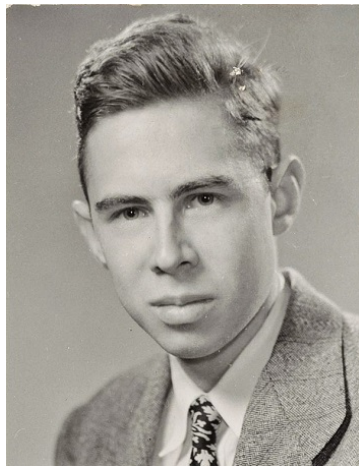
Ejemplo: un sistema experto médico típico de los años 80 contenía miles de reglas del tipo “si fiebre y tos y dolor de pecho, entonces sospechar neumonía”, escritas a mano por médicos y programadores.

El Perceptrón y los “inviernos de la IA”

En **1958**, Frank Rosenblatt creó el **Perceptrón**, el primer modelo de neurona artificial capaz de *aprender* ajustando sus propios parámetros a partir de ejemplos, en vez de tenerlos programados a mano.

La historia de la IA no ha sido un progreso constante, sino una sucesión de ciclos de entusiasmo extremo seguidos de desilusión y recortes de financiación, conocidos como “**inviernos de la IA**”:

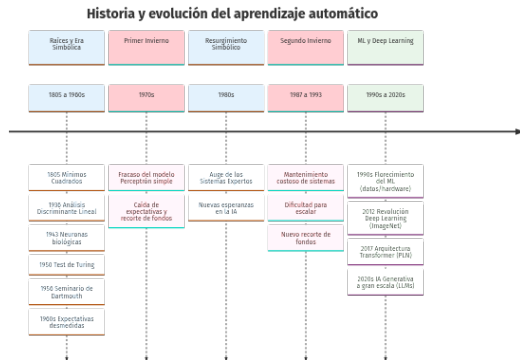
- **Primer invierno** (años 70): se descubrió que el Perceptrón, en su forma simple, no podía resolver problemas que no fueran linealmente separables.
- **Segundo invierno** (años 90): los sistemas expertos resultaron caros de mantener, difíciles de escalar y limitados a dominios muy concretos.



*Fuente: Heinz Nixdorf MuseumsForum,
CC BY-SA 4.0*

Línea temporal completa

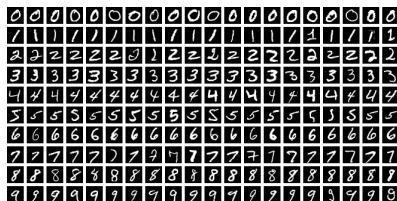
El aprendizaje automático despegó de verdad en los años 90, impulsado por hardware más rápido y por la disponibilidad de grandes cantidades de datos. A diferencia de la IA simbólica —apta para problemas lógicos como el ajedrez—, el ML permitió abordar problemas “difusos” como el reconocimiento de voz o la clasificación de imágenes.



El auge del *Deep Learning* (2010-2016)

El **Aprendizaje Profundo** (*Deep Learning*) es una rama especializada del ML que aprende **representaciones sucesivas en capas**: cada capa de la red neuronal recibe la salida de la anterior y extrae patrones cada vez más útiles para la tarea.

El punto de inflexión ocurrió en **2012**, cuando el equipo de Geoffrey Hinton ganó por un amplio margen el desafío **ImageNet** de clasificación de imágenes, usando una red neuronal profunda en lugar de los métodos estadísticos tradicionales que dominaban hasta entonces. Durante esta época, el conjunto de datos **MNIST** (dígitos manuscritos del 0 al 9) se convirtió en el banco de pruebas por excelencia para comparar algoritmos de clasificación.



Fuente: Suvanjanprasai (2024), Wikimedia Commons, CC BY-SA 4.0

La era de los LLM (2017-hoy)

A partir de **2017**, la arquitectura **Transformer** —que utiliza un mecanismo de “atención” para procesar secuencias de texto sin necesidad de capas recurrentes— desató una revolución en el procesamiento de lenguaje natural.

Esto permitió entrenar **modelos fundacionales** mediante **aprendizaje auto-supervisado** sobre cantidades ingentes de texto de internet: el modelo aprende prediciendo palabras que él mismo se oculta, sin necesidad de etiquetas creadas por humanos.

La era actual está marcada por la **IA Generativa** (como ChatGPT o Gemini), que lleva la tecnología a una escala de miles de millones de parámetros y permite no solo clasificar datos, sino **generar** contenido nuevo: texto, imágenes, código o audio, gracias a los **LLM** (*Large Language Models*).

1 Introducción

2 1.1 Historia y evolución

3 1.2 Conceptos fundamentales

Datos, *features* y etiquetas

El éxito de cualquier sistema de ML depende de cómo se represente la información. Consideremos una tabla típica:

Muestra	Feature 1	Feature 2	Feature 3	Label
1	25	70	175	Gato
2	8	4	90	Gato
3	35	80	180	Perro

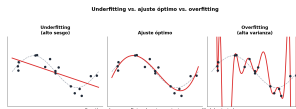
- **Muestra:** la unidad básica de información (una fila de la tabla), también llamada instancia o punto de datos.
- **Features:** los atributos que describen cada muestra (columnas de entrada).
- **Etiqueta** (*label*): la respuesta correcta asociada a cada muestra; el conjunto de todas las etiquetas constituye la “**verdad fundamental**” (*ground-truth*).

Overfitting vs. underfitting

El problema fundamental del aprendizaje automático es la tensión entre ajustar el modelo a los datos conocidos y su capacidad de funcionar bien con datos nuevos:

- **Underfitting** (subajuste): el modelo es demasiado simple para capturar el patrón real de los datos. Tanto el error de entrenamiento como el de validación son altos.
- **Overfitting** (sobreajuste): el modelo es demasiado complejo y termina memorizando el ruido del entrenamiento. El error de entrenamiento es muy bajo, pero el de validación es alto.
- El objetivo es encontrar el equilibrio **sesgo-varianza**: poco sesgo (el modelo no simplifica de más) y poca varianza (el modelo no es demasiado sensible a los datos concretos de entrenamiento).

Ejemplo: es como estudiar para un examen memorizando las respuestas de un simulacro concreto (*overfitting*) en vez de entender el tema: fallarás en cuanto cambien las preguntas.

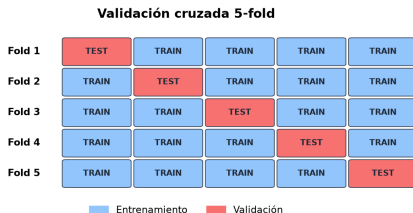


Validación cruzada *K-fold*

Para medir la generalización de forma fiable no basta con evaluar el modelo sobre los mismos datos con los que se entrenó. La práctica estándar es dividir los datos en tres conjuntos:

- **Entrenamiento:** para ajustar los pesos del modelo.
- **Validación:** para elegir los hiperparámetros y comparar modelos.
- **Prueba** (*test*): reservado para una evaluación final e imparcial.

Cuando los datos son escasos, la validación cruzada *K-fold* aprovecha mejor la información: se divide en K partes, se entrena K veces (usando cada vez una parte distinta como validación) y la métrica final es la **media** de los K resultados. La **estratificación** asegura que cada partición mantenga la misma proporción de clases que el conjunto original.



Matriz de confusión

Para evaluar un clasificador se usa una tabla llamada **matriz de confusión**, que compara las etiquetas reales con las predichas por el modelo:

Matriz de confusión

		Predicción	
		Positivo	Negativo
Clase real	Positivo	TP Verdadero Positivo	FN Falso Negativo
	Negativo	FP Falso Positivo	TN Verdadero Negativo

- **VP** (verdadero positivo): acierto positivo.
- **VN** (verdadero negativo): acierto negativo.
- **FP** (falso positivo): “falsa alarma” (error de tipo I).
- **FN** (falso negativo): “omisión” (error de tipo II).

Ejemplo: en un diagnóstico médico, un falso negativo (decir “sano” a alguien enfermo) suele ser mucho más grave que un falso positivo.

Métricas derivadas de la matriz de confusión

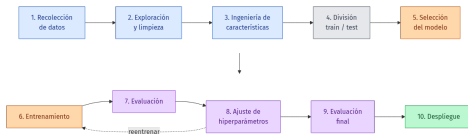
- **Exactitud** (*accuracy*): porcentaje de predicciones correctas sobre el total. Puede ser engañosa si las clases están desbalanceadas.
- **Precisión**: de todo lo que el modelo predijo como positivo, ¿cuánto acertó de verdad?
- **Recall** (sensibilidad): de todos los positivos reales que había, ¿cuántos encontró el modelo?
- **F1**: combina precisión y *recall* en un solo número; útil cuando nos importan ambas por igual.
- **AUC-ROC**: mide qué tan bien distingue el modelo los positivos de los negativos, sin depender de un umbral fijo. 1.0 es perfecto, 0.5 equivale al azar.

Ejemplo: si el 90% de los correos no son *spam*, un modelo que siempre diga “no es *spam*” tendría un 90% de exactitud sin haber aprendido nada útil: por eso a veces conviene mirar precisión y *recall* en vez de solo la exactitud.

El *pipeline* de ML: 10 etapas

El desarrollo de un proyecto de aprendizaje automático sigue un flujo de trabajo universal, desde la recolección de datos hasta el despliegue en producción. Es un proceso **iterativo**: la evaluación suele obligar a volver a etapas anteriores antes de dar el proyecto por terminado.

- ➊ Recolección de datos
- ➋ Exploración y limpieza
- ➌ Ingeniería de características
- ➍ División *train/test*
- ➎ Selección del modelo
- ➏ Entrenamiento
- ➐ Evaluación
- ➑ Ajuste de hiperparámetros
- ➒ Evaluación final
- ➓ Despliegue (y monitorización continua)



Resumen del capítulo

- El aprendizaje automático nació de una pregunta muy simple: **¿pueden las máquinas aprender?**
- Su historia no ha sido lineal: ha pasado por ciclos de entusiasmo (*hype*) e “inviernos” de desilusión y recortes.
- El estado del arte actual son el **aprendizaje profundo** y los **LLM**, posibles gracias a más datos, mejor hardware y algoritmos más eficientes.
- **Muestra**, **feature** y **etiqueta** son las piezas básicas de cualquier conjunto de datos.
- El **overfitting/underfitting** y la **validación cruzada** son las herramientas para saber si un modelo generaliza bien.
- La **matriz de confusión** y sus métricas derivadas permiten elegir la medida de éxito adecuada según el problema.
- El **pipeline de ML** organiza todo el proceso en 10 etapas iterativas, de los datos al despliegue.

Capítulo 1: Fundamentos del Aprendizaje Automático

Universidad de Salamanca

Facultad de Ciencias

Programación Avanzada

Diego M. Jiménez-Bravo · dmjimenez@usal.es